Mohaiya aldossary
Student ID: 2230004579

In [1]:
```
%pip install -q pandas scikit-learn nltk
```

```
Note: you may need to restart the kernel to use updated packages.
[notice] A new release of pip is available: 23.2.1 -> 26.2.1
[notice] To update, run: C:\Users\ezio9\OneDrive\Desktop\IAU\NLP\NLP lab\.lab_work\venv\Scripts\python.exe -m pip install --upgrade pip
```

Classification and Evaluation in NLP

Text classification is a common Natural Language Processing (NLP) task where a model assigns a predefined category or label to a piece of text.

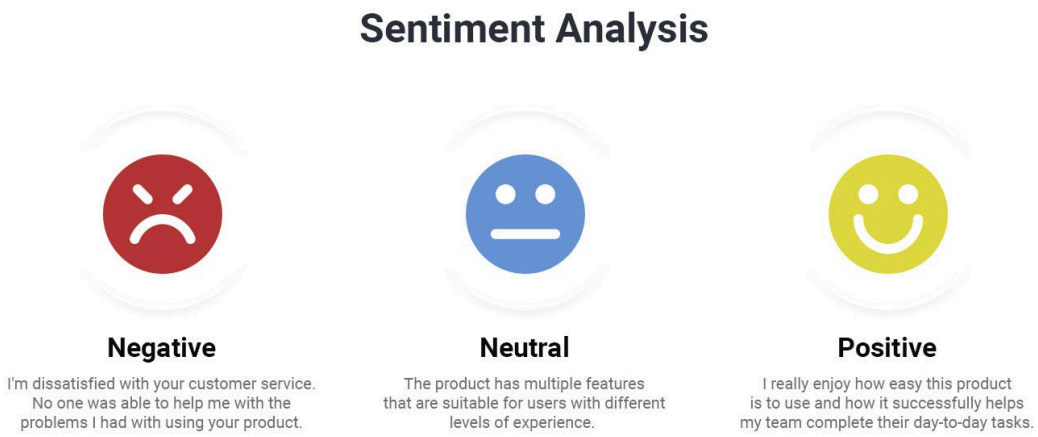
Examples include:

- Spam detection
- Sentiment analysis
- News categorization

In this lab, we will learn how to build and evaluate a text classification model.

Case Study: Sentiment Analysis

Sentiment analysis, also known as opinion mining, its goal is to understand the information present in the text and categorize it as positive, negative, or neutral.



Disneyland Reviews Dataset

In this lab we'll look into Disneyland Reviews dataset.
Link: https://www.kaggle.com/datasets/arushchillar/disneyland-reviews

The dataset includes 42,000 reviews of 3 Disneyland branches - Paris, California and Hong Kong, posted by visitors on Trip Advisor.

Column Description:

- Review_ID: unique id given to each review
- Rating: ranging from 1 (unsatisfied) to 5 (satisfied)
- Year_Month: when the reviewer visited the theme park
- Reviewer_Location: country of origin of visitor
- Review_Text: comments made by visitor
- Disneyland_Branch: location of Disneyland Park

In [2]:

```
# Load the dataset
import pandas as pd

data = pd.read_csv('data/DisneylandReviews.csv', encoding='latin-1')
data
```

Out[2]:

	Review_ID	Rating	Year_Month	Reviewer_Location	Review_Text	Branch
0	670772142	4	2019-4	Australia	If you've ever been to Disneyland anywhere you...	Disneyland_HongKong
1	670682799	4	2019-5	Philippines	Its been a while since d last time we visit HK...	Disneyland_HongKong
2	670623270	4	2019-4	United Arab Emirates	Thanks God it wasn t too hot or too humid wh...	Disneyland_HongKong
3	670607911	4	2019-4	Australia	HK Disneyland is a great compact park. Unfortu...	Disneyland_HongKong
4	670607296	4	2019-4	United Kingdom	the location is not in the city, took around 1...	Disneyland_HongKong
...	...	...	...	...	...	...
42651	1765031	5	missing	United Kingdom	i went to disneyland paris in july 03 and thou...	Disneyland_Paris
42652	1659553	5	missing	Canada	2 adults and 1 child of 11 visited Disneyland ...	Disneyland_Paris
42653	1645894	5	missing	South Africa	My eleven year old daughter and myself went to...	Disneyland_Paris
42654	1618637	4	missing	United States	This hotel, part of the Disneyland Paris compl...	Disneyland_Paris
42655	1536786	4	missing	United Kingdom	I went to the Disneyparis resort, in 1996, wit...	Disneyland_Paris

42656 rows × 6 columns

In [3]:

```
#Check for missing values
data.isna().sum()
```

Out[3]:

```
Review_ID           0
Rating              0
Year_Month          0
Reviewer_Location   0
Review_Text         0
Branch              0
dtype: int64
```

In [4]:

```
# Drop rows with missing values in the relevant columns
data = data.dropna(subset=['Review_Text', 'Rating'])
```

In [5]:

```
# Map star ratings to sentiments
data['Sentiment'] = data['Rating'].apply(lambda rating: 'positive' if rating > 3 else ('negative' if rating < 3 else
'neutral'))
data.head()
```

Out[5]:

	Review_ID	Rating	Year_Month	Reviewer_Location	Review_Text	Branch	Sentiment
0	670772142	4	2019-4	Australia	If you've ever been to Disneyland anywhere you...	Disneyland_HongKong	positive
1	670682799	4	2019-5	Philippines	Its been a while since d last time we visit HK...	Disneyland_HongKong	positive
2	670623270	4	2019-4	United Arab Emirates	Thanks God it wasn t too hot or too humid wh...	Disneyland_HongKong	positive
3	670607911	4	2019-4	Australia	HK Disneyland is a great compact park. Unfortu...	Disneyland_HongKong	positive
4	670607296	4	2019-4	United Kingdom	the location is not in the city, took around 1...	Disneyland_HongKong	positive

In [6]:

```
# Import necessary libraries
import re
import nltk

def preprocess_text(text):

    # 1. Convert text to lowercase
    text = text.lower()

    # 2. Remove punctuation and special characters
    text = re.sub(r'[^a-z\s]', '', text)

    # 3. Tokenize the text
    words = text.split()

    return ' '.join(words)

# Apply preprocessing to the review text
data['Review_Text'] = data['Review_Text'].apply(preprocess_text)

# Display cleaned reviews
data[['Review_Text', 'Sentiment']].head()
```

Out[6]:

	Review_Text	Sentiment
0	if youve ever been to disneyland anywhere youl...	positive
1	its been a while since d last time we visit hk...	positive
2	thanks god it wasn t too hot or too humid when...	positive
3	hk disneyland is a great compact park unfortun...	positive
4	the location is not in the city took around ho...	positive

In [7]:

```
# Import necessary libraries
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

# Split the dataset into training and testing sets
train_data, test_data, train_labels, test_labels = train_test_split(data['Review_Text'], data['Sentiment'], test_size=0.2,
random_state=42)


# TF-IDF Vectorization
tfidf_vectorizer = TfidfVectorizer(max_features=1000)  # You can adjust max_features based on your dataset size - limit
vocabulary to the 1000 most informative tokens
train_vectors = tfidf_vectorizer.fit_transform(train_data)
test_vectors = tfidf_vectorizer.transform(test_data)
```

In [8]:

```
print(f"train_data size: {train_data.shape}")
print(f"test_data size: {test_data.shape}")
```

```
train_data size: (34124,)
test_data size: (8532,)
```

In [9]:

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Train the Support Vector Machine (SVM) classifier
svm_classifier = SVC(kernel='linear')
svm_classifier.fit(train_vectors, train_labels)

# Predictions on the test set
predictions = svm_classifier.predict(test_vectors)

# Evaluate the model
accuracy = accuracy_score(test_labels, predictions)
print(f"Accuracy: {accuracy:.2f}")

# Display classification report
print("Classification Report:")
print(classification_report(test_labels, predictions))
```

```
Accuracy: 0.84
Classification Report:
              precision    recall  f1-score   support

    negative       0.64      0.54      0.58       720
     neutral       0.48      0.18      0.26      1035
    positive       0.88      0.98      0.92      6777

    accuracy                           0.84      8532
   macro avg       0.66      0.56      0.59      8532
weighted avg       0.81      0.84      0.81      8532
```

Use Case: Amazon reviews of unlocked phone analysis

The objective of this use case is to conduct sentiment analysis on Amazon reviews of unlocked phones, categorizing reviews into three classes: positive, negative, and neutral. The goal is to gain a comprehensive understanding of customer opinions and sentiments regarding various unlocked phone models.

- Dataset link: https://www.kaggle.com/datasets/PromptCloudHQ/amazon-reviews-unlocked-mobile-phones

Task1: Load the data and do 5 different preprocessing steps on it

Task2: Use a proper train and test split

Task3: Do Feature Extraction Using TF-IDF

Task4: Train a Naive Bayes Classifier

Task5: Evaluation the model on the Test Set

Task6: Print the Confusion Matrix

In [10]:

```
# Task1
import pandas as pd
import re
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

amazon_data = pd.read_csv('data/Amazon_Unlocked_Mobile.csv')
amazon_data = amazon_data.dropna(subset=['Reviews', 'Rating']).copy()
amazon_data['Sentiment'] = amazon_data['Rating'].apply(
    lambda rating: 'positive' if rating > 3 else ('negative' if rating < 3 else 'neutral')
)
stemmer = PorterStemmer()
stop_words = set(ENGLISH_STOP_WORDS) - {'no', 'not', 'nor', 'never'}

def preprocess_review(text):
    text = text.lower()  # 1. Lowercase
    text = re.sub(r'[^a-z\s]', ' ', text)  # 2. Remove punctuation and numbers
    tokens = text.split()  # 3. Tokenize
    tokens = [word for word in tokens if word not in stop_words]  # 4. Remove stopwords
    tokens = [stemmer.stem(word) for word in tokens]  # 5. Stem
    return ' '.join(tokens)

amazon_data['Cleaned_Review'] = amazon_data['Reviews'].apply(preprocess_review)
amazon_data = amazon_data[amazon_data['Cleaned_Review'].str.strip().ne('')]
amazon_data = amazon_data.drop_duplicates(subset=['Cleaned_Review'])
amazon_data[['Reviews', 'Cleaned_Review', 'Sentiment']].head()
```

Out[10]:

	Reviews	Cleaned_Review	Sentiment
0	I feel so LUCKY to have found this used (phone...	feel lucki use phone not use hard phone line u...	positive
1	nice phone, nice up grade from my pantach revu...	nice phone nice grade pantach revu clean set e...	positive
2	Very pleased	pleas	positive
3	It works good but it goes slow sometimes but i...	work good goe slow good phone love	positive
4	Great phone to replace my lost phone. The only...	great phone replac lost phone thing volum butt...	positive

In [11]:

```
# Task2
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    amazon_data['Cleaned_Review'], amazon_data['Sentiment'],
    test_size=0.2, random_state=42, stratify=amazon_data['Sentiment']
)
print(f'Training reviews: {len(X_train)}')
print(f'Test reviews: {len(X_test)}')
```

```
Training reviews: 121276
Test reviews: 30320
```

In [12]:

```
# Task3
from sklearn.feature_extraction.text import TfidfVectorizer

amazon_vectorizer = TfidfVectorizer(max_features=10000, ngram_range=(1, 2))
X_train_tfidf = amazon_vectorizer.fit_transform(X_train)
X_test_tfidf = amazon_vectorizer.transform(X_test)
print(f'Training TF-IDF shape: {X_train_tfidf.shape}')
print(f'Test TF-IDF shape: {X_test_tfidf.shape}')
```

```
Training TF-IDF shape: (121276, 10000)
Test TF-IDF shape: (30320, 10000)
```

In [13]:

```
# Task4
from sklearn.naive_bayes import MultinomialNB

nb_classifier = MultinomialNB()
nb_classifier.fit(X_train_tfidf, y_train)
```

Out[13]:

MultinomialNB

▶ Parameters

In [14]:

```
# Task5
from sklearn.metrics import accuracy_score, classification_report

amazon_predictions = nb_classifier.predict(X_test_tfidf)
print(f'Accuracy: {accuracy_score(y_test, amazon_predictions):.4f}')
print('Classification Report:')
print(classification_report(y_test, amazon_predictions, zero_division=0))
```

```
Accuracy: 0.8273
Classification Report:
              precision    recall  f1-score   support

    negative       0.78      0.81      0.80      8639
     neutral       0.40      0.04      0.07      2794
    positive       0.85      0.95      0.90     18887

    accuracy                           0.83     30320
   macro avg       0.68      0.60      0.59     30320
weighted avg       0.79      0.83      0.79     30320
```

In [15]:

```
# Task6
from sklearn.metrics import confusion_matrix

sentiment_labels = ['negative', 'neutral', 'positive']
matrix = confusion_matrix(y_test, amazon_predictions, labels=sentiment_labels)
print('Confusion Matrix:')
print(pd.DataFrame(
    matrix,
    index=pd.Index(sentiment_labels, name='Actual'),
    columns=pd.Index(sentiment_labels, name='Predicted')
))
```

```
Confusion Matrix:
Predicted  negative  neutral  positive
Actual
negative       7013       86      1540
neutral        1079      104      1611
positive        850       71     17966
```